# Desafío 1: Implementación de Tetris mediante manipulación de bits en C++

**Integrantes del grupo:** Clayder Areiza Pino & Arlington Zahir Llerena

CC: 10421450543 & CC: 172 277 711

**Email:** clayder.areiza@udea.edu.co, zahir.llerena@udea.edu.co

**Número telefónico:** + 57 310 726 12 82

**Profesores:** AUGUSTO ENRIQUE SALAZAR JIMENEZ & KEVIN DAVID MARTINEZ ZAPATA

**Grupo 003. Código: 2570201 – Informática II -**

**Facultad de Ingeniería en Telecomunicaciones Región – Yarumal & Carmen del Viboral -**

**Universidad de Antioquia**

**20 de marzo de 2026**

## Objetivos técnico general

- **Desarrollar un motor de Tetris por consola donde la lógica central (movimiento, colisiones, rotaciones y tablero) deban basarse estrictamente en operadores a nivel de bits y memoria dinámica. Se prohíbe el uso de la STL, el tipo string y sintaxis de ANSI C.**

## Objetivos específicos

- ***Gestión Eficiente de Memoria*: Implementar el tablero y las piezas mediante un arreglo de punteros a datos enteros, utilizando cada bit como una unidad mínima de información para optimizar el espacio ocupado en la memoria del equipo.**
- ***Lógica Bitwise*: Desarrollar los algoritmos de colisión, movimiento y rotación empleando exclusivamente operaciones lógicas (AND, OR, XOR) y desplazamientos, eliminando el procesamiento basado en matrices.**

- ***Modularidad Estricta*: Construir script con extensiones dadas por archivos (.h y .cpp) sin dependencias de la STL o tipos string, garantizando el control absoluto sobre la memoria y los flujos de datos base del lenguaje.**

**Propuesta preliminar desarrollo.**

**Antes de iniciar cualquier proyecto ingenieril, independiente de la disciplina de profundización. Planteamos unas preguntas que permiten direccionar la propuesta y además nos brinda la oportunidad de esclarecer conceptos vinculados con ejercicios de programación en el lenguaje C++. A continuación, damos a conocer las preguntas que desde nuestra opinión establecen la ruta de trabajo.**

**Fase 1: Estructuras de Datos y Memoria (Nivel Base)**

- **Representación Binaria del Tablero: Si el ancho del tablero debe ser múltiplo de 8, ¿qué tipo de dato base de C++ elegirías para representar una fila de bits de manera que optimices el uso de memoria y facilites los desplazamientos laterales?**
- **Gestión Dinámica: Dado que las dimensiones del tablero las define el usuario al inicio, ¿cómo implementarías la reserva de memoria para el tablero utilizando punteros para asegurar que la estructura sea flexible pero eficiente?**
- **Restricción de Tipos: Considerando que no se permiten objetos "string" ni la STL, ¿cómo diseñarías una concatenación de caracteres para la visualización en consola que traduzca los bits (0 y 1) en caracteres visuales como, por ejemplo: (' ' y '#')?**

**Fase 2: Lógica de Manipulación de Bits (Nivel Intermedio)**

- **Modelado: Para representar las piezas de hasta 3 x 2 o 2 x 3, ¿qué ventajas ofrece usar una máscara de bits (mask) en lugar de un arreglo bidimensional convencional?**
- **Detección de Colisiones: Utilizando exclusivamente operadores & (AND) y desplazamientos << o >>, ¿cuál serían la expresión lógica para verificar si una pieza en una posición (x, y) choca con un bloque ya existente en el tablero o con los límites laterales?**
- **Mecánica de Rotación: Si las rotaciones deben seguir patrones específicos, ¿cómo podrías recalcular estas rotaciones como constantes de bits para evitar cálculos complejos en tiempo de ejecución?**

**Fase 3: Algoritmos de Juego y Optimización (Nivel Avanzado)**

- **Limpieza de Filas:** Cuando una fila tiene todos sus bits en 1, ¿cómo podrías usar la aritmética de punteros y operaciones de asignación de bits para "desplazar" hacia abajo todas las filas superiores de manera eficiente?
- **Detección de Game Over:** ¿Cuál es el criterio matemático -bit a bit- para determinar que una pieza nueva no puede ser colocada en el origen (ubicación central) del tablero?
- **Arquitectura Modular:** Dado que el proyecto exige el uso de archivos .h y .cpp, ¿cómo separarías la lógica del "Motor de Bits" de la "Interfaz de Consola" para cumplir con los principios de programación estructurada solicitados?

Aunque las preguntas anteriores, pueden dar la impresión de ser triviales, las respuestas nos permiten dimensionar la complejidad de la iniciativa. Así mismo, nos brindan pautas de que elementos son los primarios para realizar una propuesta consecuente con los requerimientos puntuales del reto. Atendiendo a lo comentado por el momento, a continuación, damos a conocer una propuesta general que hemos pensado para la solución del reto. Por supuesto, es una propuesta preliminar que probablemente cambien en la hora de llevar a cabo la construcción de los algoritmos por módulos.

## 1. Propuesta: El Enfoque de "Bits Estático"

Esta propuesta se centra en ver el tablero como una estructura de datos de tamaño dinámica en bits para acelerar las operaciones lógicas.

- **Estructura del Tablero:** Tratar cada fila del tablero de Tetris como un número entero único. En lugar de ver el tablero como una cuadrícula compleja, lo vemos como una columna de números.
- **Representación de Piezas:** Cada pieza se almacena como una "máscara" (un número entero). Por ejemplo, una pieza de 2 x 2 es simplemente el número 3 en dos filas consecutivas (binario 00000011).
- **Lógica de Movimiento:** Para mover una pieza a la derecha, aplicar un desplazamiento de bits a la derecha (>>) sobre la máscara de la pieza antes de compararla con el tablero.
- **Detección de Colisiones:** Es la más rápida de implementar. Realizar un AND lógico entre la fila del tablero y la máscara de la pieza. Si el resultado es > 0, hay colisión.

En términos generales, el presente proyecto se fundamenta en el paradigma de la **programación estructurada y modular**. En lugar de crear un código lineal, la solución

se fragmenta en "unidades de trabajo" que definen “funciones específicas”. Esto permite que el sistema sea fácil de depurar, eficiente en el uso de memoria y técnicamente elegante, además nos brinda la oportunidad de gestionar los errores de manera puntual, dirigiéndonos al fragmento de código del cual se sospecha que pueda no ejecutarse de manera adecuada.

La arquitectura se basa en el uso predominante de funciones de tipo void(). En programación, una función void() es una instrucción que realiza una **acción concreta** sobre el estado del juego (como dibujar, fijar o limpiar) sin necesidad de devolver un valor simple, sino modificando directamente las estructuras de datos (el tablero) a través de parámetros.

**Componentes Esenciales (Las Tareas)**

El proyecto se divide en cinco pilares funcionales, donde cada función tiene una "responsabilidad":

- **Gestión de Memoria y Dimensiones:** La función inicial se encarga de preparar el escenario. Al permitir que el usuario defina el alto y ancho, el programa utiliza punteros para reservar solo el espacio necesario, garantizando que no se desperdicie ni un solo byte de la memoria RAM.
- **Motor de Generación Estocástica:** Mediante una función dedicada, el sistema selecciona aleatoriamente uno de los 7 tetriminos clásicos. Se utiliza un generador moderno de números para asegurar que el juego sea impredecible y justo.
- **Núcleo de Colisiones (Lógica Bitwise):** Esta es la inteligencia del juego. Mediante operaciones matemáticas de bits (AND lógico), el programa verifica en microsegundos si una pieza toca los bordes o bloques ya fijados. Es la parte más eficiente del código, ya que procesa filas enteras como una sola unidad numérica.
- **Transformación y Movimiento:** Funciones específicas se encargan de los desplazamientos laterales y la rotación. La rotación es un proceso crítico que reasigna los bits de la pieza para cambiar su orientación espacial, validando siempre la integridad antes de confirmar el movimiento.
- **Sistema de Gravedad y Limpieza:** Cuando una fila se completa (todos sus bits están en 1), una función de limpieza "comprime" el tablero, eliminando esa fila y desplazando las superiores hacia abajo, emulando la física de gravedad del juego original.

**Flujo de Ejecución (El Ciclo de Vida)**

**El proyecto sigue un orden lógico de ejecución que garantiza la estabilidad del software:**

- **Configuración:** El usuario define el entorno.
- **Nacimiento:** Se crea una pieza nueva en el tope.
- **Interacción:** El usuario envía comandos (A, D, S, W) que activan las funciones modulares.
- **Validación:** Cada movimiento es filtrado por la función de colisión.
- **Persistencia:** Al tocar fondo, la pieza se "fija" y se revisa la puntuación (limpieza de filas).
- **Finalización:** El sistema libera la memoria dinámica al terminar, asegurando un cierre limpio del programa.

**Conclusión Pedagógica**

**Este proyecto demuestra que la complejidad de un juego puede reducirse a una serie de *tareas simples y coordinadas*. El uso de funciones modulares no solo cumple con los requisitos académicos, sino que nos prepara a nosotros como futuros ingenieros en proyectos de software a gran escala, donde la claridad y la reutilización del código son fundamentales.**

**Bibliografía.**


- **Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley. (2010).**
- **Deitel, P. J., & Deitel, H. M. (2017). *C++ How to Program* (10th ed.). Pearson.**
- **Prata, S. (2011). *C++ Primer Plus* (6th ed.). Addison-Wesley.**


UdeA